save code in ActiveRecord::Base by calling super. If the validation fails, then it returns false. Validation#save! follows the same steps, except that it raises an exception if the validation fails.

These days, Rails barely ever uses alias_method_chain. You can still find this method called inside Active Support and some third-party libraries, but there is no trace of it in libraries such as Active Record. The once-popular alias_method_chain has nearly disappeared from the Rails environment.

However, there is still one case where you might argue that alias_method_chain works better than its object-oriented alternative. Let's look at it.

The Birth of Module#prepend

Let's add a twist to our ongoing greet method example: instead of defining greet in a module, let's assume it's defined directly in the class.

part2/greet_with_prepend.rb

```
class MyClass
  def greet
    "hello"
  end
end

MyClass.new.greet  # => "hello"
```

In this case, you cannot wrap functionality around greet by simply including a module that overrides it:

part2/greet_with_prepend_broken.rb

```
module EnthusiasticGreetings
  def greet
    "Hey, #{super}!"
  end
end

class MyClass
  include EnthusiasticGreetings
end

MyClass.ancestors[0..2]  # => [MyClass, EnthusiasticGreetings, Object]
MyClass.new.greet        # => "hello"
```

The code above shows that when you include EnthusiasticGreetings, that module gets higher than the class in the class's chain of ancestors. As a result, the greet method in the class overrides the greet method in the module, instead of the other way around.